

The Curry-Howard Correspondence for Mathematicians

Daniel Schepler

December 25, 2025

1 Introduction

In my experience, many presentations of the Curry-Howard correspondence seem to be targeted either to computer scientists or to specialists in mathematical and formal logic. The aim of this article is to provide an introduction which should be accessible to mathematicians and mathematics students in general.

2 Beginning Observations

To begin with, we will make a very simple, almost trivial observation:

Proposition 1. *Proposition: Let p be a proposition, i.e. a mathematical statement which is either true or false. Then there exists a set S such that p is equivalent to $S \neq \emptyset$.*

Proof. If p is true, then $S := \{0\}$ works; if p is false, then $S := \emptyset$ works. $\square$

The next step will be to make some observations which will allow us to greatly refine the above construction into something which is more functorial in nature.

- Proposition 2.**
1. *If S and T are sets, then $(S \neq \emptyset) \wedge (T \neq \emptyset)$ if and only if $S \times T \neq \emptyset$.*
 2. *If S and T are sets, then $(S \neq \emptyset) \vee (T \neq \emptyset)$ if and only if $S \sqcup T \neq \emptyset$.*
 3. *If S and T are sets, then $(S \neq \emptyset) \rightarrow (T \neq \emptyset)$ if and only if $T^S \neq \emptyset$, where T^S is the set of functions $f : S \rightarrow T$.*
 4. *In particular, if S is a set, then $\neg(S \neq \emptyset)$ if and only if $\emptyset^S \neq \emptyset$.*
 5. *If $(S_i)_{i \in I}$ is an indexed family of sets, then $\forall i \in I, S_i \neq \emptyset$ if and only if $\prod_{i \in I} S_i \neq \emptyset$.*

6. If $(S_i)_{i \in I}$ is an indexed family of sets, then $\exists i \in I, S_i \neq \emptyset$ if and only if $\bigsqcup_{i \in I} S_i \neq \emptyset$.

Note that in 2, we could also have used $S \cup T$ instead of $S \sqcup T$. The reason we choose $S \sqcup T$, as we will see later, is that it is easier to define functions with domain $S \sqcup T$ than it is to define functions with domain $S \cup T$. Similarly in 6, we could have used $\bigcup_{i \in I} S_i$, but again, it is easier to define functions on $\bigsqcup_{i \in I} S_i$ than on $\bigcup_{i \in I} S_i$.

- Proof.* 1. $(\Rightarrow)$: Choose $x \in S$ and $y \in T$. Then $(x, y) \in S \times T$. $(\Leftarrow)$: Suppose $p = (x, y) \in S \times T$; then $x \in S$ and $y \in T$.
2. $(\Rightarrow)$: For case 1, if $S \neq \text{emptyset}$, then choose $x \in S$; then also $x \in S \sqcup T$. For case 2, if $T \neq \emptyset$, then choose $y \in T$; then also $y \in S \sqcup T$. $(\Leftarrow)$: If $x \in S \sqcup T$, then either $x \in S$ or $x \in T$. In the first case, we get $S \neq \emptyset$; in the second case, we get $T \neq \emptyset$.
3. $(\Rightarrow)$: Suppose $(S \neq \emptyset) \rightarrow (T \neq \emptyset)$. Then in case $S = \emptyset$, then the empty function is always in T^S . Otherwise, if $S \neq \emptyset$, then also $T \neq \emptyset$; choosing $y \in T$, then the constant function sending every element of S to y is in T^S . $(\Leftarrow)$: Suppose $T^S \neq \emptyset$ and $S \neq \emptyset$. Then choosing $f \in T^S$ and $x \in S$, we have $f(x) \in T$, so $T \neq \emptyset$.
4. This follows by the observation that $\neg P$ is equivalent to $P \rightarrow \perp$ where $\perp$ is the false proposition, and $\perp$ in turn is equivalent to $\emptyset \neq \emptyset$.
5. $(\Rightarrow)$: This implication is precisely one of the standard versions of the axiom of choice. $(\Leftarrow)$: Suppose $(x_i)_{i \in I} \in \prod_{i \in I} S_i$. Then for any $i \in I$, we have $x_i \in S_i$, so $S_i \neq \emptyset$.
6. $(\Rightarrow)$: Suppose $\exists i \in I, S_i \neq \emptyset$. Then choose some such i , and for that i , choose $x \in S_i$. Then $x \in \bigsqcup_{i \in I} S_i$ also. $(\Leftarrow)$: Suppose $\bigsqcup_{i \in I} S_i \neq \emptyset$, and choose $x \in \bigsqcup_{i \in I} S_i$. Then $x \in S_i$ for some $i \in I$, and for this i , we conclude $S_i \neq \emptyset$.

□

This allows us to refine the original observation as follows: first, take your proposition and break it down as much as possible into combinations of atomic propositions via the standard logical connectives. Then, for each atomic proposition, choose a set whose nonemptiness is equivalent to that proposition, and recursively use the last result to build up a set whose nonemptiness is equivalent to the compound proposition.

Example 1. Let us take the proposition $(p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))$. Here, the atomic propositions are p, q, r ; let us select sets A, B, C such that p is equivalent to $A \neq \emptyset$, q is equivalent to $B \neq \emptyset$, and r is equivalent to $C \neq \emptyset$. In order to avoid deeply nested exponential notation, let us use the notation $X \multimap Y$ in place of Y^X . Then by the above, the proposition is equivalent to $(A \multimap (B \multimap C)) \multimap ((A \multimap B) \multimap (A \multimap C))$ being nonempty.

3 The Curry-Howard Correspondence

One half of the Curry-Howard correspondence is this easy observation: if you can explicitly name an element of the corresponding set, then that automatically gives a proof of the desired proposition.

Example 2. For any two sets A, B , we have the projection $\pi_1 : A \times B \rightarrow A$. Therefore, $A^{A \times B} \neq \emptyset$. If $A \neq \emptyset$ is equivalent to p and $B \neq \emptyset$ is equivalent to q , this shows that $p \wedge q \rightarrow p$ is true. Since this holds regardless of whether A, B are nonempty, this shows that $p \wedge q \rightarrow p$ is a tautology.

Example 3. If we have $f \in A \multimap (B \multimap C)$, $g \in A \multimap B$, and $x \in A$, then $f(x) \in B \multimap C$, and $g(x) \in B$, so $f(x)(g(x)) \in C$. Abstracting this into functions, we can construct

$$(f \mapsto (g \mapsto (x \mapsto f(x)(g(x))))) \in (A \multimap (B \multimap C)) \multimap ((A \multimap B) \multimap (A \multimap C)).$$

Since this holds regardless of whether A, B, C are nonempty, this shows that $(p \rightarrow (q \rightarrow r)) \rightarrow ((p \rightarrow q) \rightarrow (p \rightarrow r))$ is a tautology.

Example 4. Say we have a family of sets $(S_n)_{n=0}^\infty$ indexed by $\mathbb{N}$. Then given an element $x_0 \in S_0$, and a family of functions $f_n : S_n \rightarrow S_{n+1}$ for each $n \in \mathbb{N}$, we can recursively define $x_n \in S_n$ such that x_0 is as specified, and $x_{n+1} = f_n(x_n)$. This construction gives an element of $(S_0 \times \prod_{n=0}^\infty (S_n \multimap S_{n+1})) \multimap \prod_{n=0}^\infty S_n$. Now, if p_n is a family of propositions indexed by $\mathbb{N}$, such that p_n is equivalent to $S_n \neq \emptyset$, then the corresponding proposition we have proved is: $p_0 \rightarrow [(\forall n \in \mathbb{N}, p_n \rightarrow p_{n+1}) \rightarrow \forall n \in \mathbb{N}, p_n]$. This is exactly the principle of induction.

The other direction of the Curry-Howard correspondence is: given a proof of a proposition, it is often straightforward to extract an explicit element of the corresponding set. This is especially the case if the proof is formalized in a natural deduction proof system, using correspondences between the rules of such a system and basic building blocks of terms in the corresponding sets.

Example 5. Let us take the natural deduction rule $\rightarrow I$, which states: suppose Γ is some context of propositions which are assumed to be true. Then if $\Gamma \cup \{p\} \vdash q$, i.e. when adding p to the context you can prove q , then $\Gamma \vdash (p \rightarrow q)$, i.e. in the original context you have proved that $p \rightarrow q$ is true. This rule can be thought of as “conditional proof”: to prove an implication proposition, you temporarily assume the left hand side and prove the right hand side.

Now, let us suppose you have built up a term β representing an element of T , possibly with some free variables assumed to be elements of sets corresponding to the propositions of Γ , along with a “new” free variable x assumed to be in S . Then the abstracted function $x \mapsto \beta$ is a term representing an element of T^S . In other words, in the Curry-Howard correspondence, the proof rule $\rightarrow I$ corresponds to function abstraction — also known in computer science or type theory as the λ operator, as in lambda calculus.

Example 6. Let us take the natural deduction rule $\vee E$, which states: suppose Γ is some context of propositions which are assumed to be true. Also, suppose $\Gamma \vdash p \vee q$, i.e. from Γ you can prove that $p \vee q$ is true; $\Gamma \cup \{p\} \vdash r$; and $\Gamma \cup \{q\} \vdash r$. Then $\Gamma \vdash r$. This is a formalization of the method of proof by cases: if you can prove $p \vee q$, and then you can split this into cases that p is sufficient to establish what you want, and also that q is sufficient to establish what you want, then that concludes the proof.

Now, let us suppose you have built up a term δ representing an element of $S \sqcup T$, possibly with some free variables assumed to be in sets corresponding to the propositions in Γ . Also, suppose you have a term α with a possible additional free variable x assumed to be in S , where α represents an element of U ; and similarly, you have a term β with a possible additional free variable y assumed to be in T , and β also represents an element of U . Then $x \mapsto \alpha$ represents an element of U^S , and $y \mapsto \beta$ represents an element of U^T , so by the universal property of $S \sqcup T$ as a coproduct set of S and T , we have $(x \mapsto \alpha) + (y \mapsto \beta)$ represents an element of $U^{S \sqcup T}$. Therefore, $[(x \mapsto \alpha) + (y \mapsto \beta)](\delta)$ represents an element of U , using only the original free variables corresponding to propositions in Γ .

With a little programming background, this construction may be a bit easier to understand as follows: the disjoint union of two types A and B can be viewed as a tagged union type, where each element contains either a tag **left** along with an element of A , or a tag **right** along with an element of B . Then the construction above might be represented in a typical functional programming language as: **match** δ **with** **left** $x \Rightarrow \alpha$ | **right** $y \Rightarrow \beta$ **endmatch**.

There are a couple notable points which are trickier. One is proofs involving equality. One way to handle this is: suppose x and y are in some larger set S . Then we can define E_x as a family of sets indexed by S , where $(E_x)_x$ has one element ρ_x , and otherwise $(E_x)_y$ is empty. In this case, the proof rule $=I$ that states $x = x$ corresponds to the term ρ_x . On the other hand, the proof rule $=E$, otherwise known as the substitution principle, can be used to define a corresponding conversion operator in $T_x \multimap \prod_{y \in S} ((E_x)_y \multimap T_y)$. This does tend to end up producing terms that are somewhat harder than the others to understand and to reason with, however.

The more serious point revolves around the proof rule of double negation elimination, which is needed for classical logic. This proof rule would correspond to functions in $((S \multimap \emptyset) \multimap \emptyset) \multimap S$. However, even when S is nonempty, $(S \multimap \emptyset) \multimap \emptyset$ has a single element, so specifying such a function for each set S would amount to a global choice operator, or a Hilbert epsilon operator. This is not something that can be constructed in the standard axiomatic set theory of ZFC (although introducing a Hilbert epsilon operator turns out to be a conservative extension, so it does not introduce any inconsistency to add it).

An alternative is to observe that classical logic can also be produced using Peirce's law, $((p \rightarrow q) \rightarrow p) \rightarrow p$. The corresponding set $((S \multimap T) \multimap S) \multimap S$ is exactly the type of a functional programming construct known as *call with current continuation*, which essentially allows a programmer to save an execu-

tion checkpoint, and then later rewind execution to that checkpoint but with a different return value specified. Adding this construct to a functional programming language introduces considerable issues with reasoning about programs, however. For example, $f(x) == f(x)$ no longer necessarily returns `true`, if one execution of f has had some rewinding happen which the other has not.

Given the issues mentioned above with terms corresponding to proofs in classical logic, it would be a natural question to ask which proofs correspond to the terms which avoid Hilbert epsilon and call with current continuation. It turns out that this gives first-order intuitionistic logic, which therefore becomes of interest to computer scientists.